

Reviewer Pack — Minimal Rank of Tropicalized Configuration Spaces vs ACC^0 Ci...

Entry 7d02dff3d32a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 16:26:52 UTC

Minimal Rank of Tropicalized Configuration Spaces vs ACC^0 Circuit Threshold for Tensor Product

Entry ID: 7d02dff3d32a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 16:26:52 UTC

1. Conjecture

Field A (mathematical branch): Configuration Space Theory **Field B** (complexity object): Complexity Theory: ACC^0 Circuit Complexity

Statement:

[‘For all instances of the Tensor Product problem with input size n , there exists a tropicalization of the configuration space that has a minimal rank bounded by $O(n^2 \log n)$. This implies that the ACC^0 circuit complexity for Tensor Product is at least $O(n^2 \log n)$.’, ‘Equivalently, for any ACC^0 circuit C computing Tensor Product, its size must be at least $\Theta(n^2 \log n)$, where the runtime of the computation is measured in tropical operations.’, ‘Specifically, there exists a polynomial-time constructive mapping from an instance of Tensor Product to a configuration space with the above properties.’]

Rationale (proposer’s reasoning):

[‘Configuration Space Theory provides a geometric perspective on computational problems, which can potentially expose hidden structures that are not apparent through traditional complexity-theoretic approaches.’, ‘The connection between tropicalization and ACC^0 circuit complexity may reveal new insights into the limitations of classical computation models and the potential for quantum-inspired algorithms.’]

Taxonomy category: CONFIG_SPACE_TO_ACC0 (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 15691ec93fb05956

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the instances with $n \leq 40$ have a tropicalized configuration space with a minimal rank not exceeding $O(n^2 \log n)$ and an ACC^0 circuit complexity that meets or exceeds the lower bound of $O(n^2 \log n)$. The conjecture is falsified if any instance has a tropicalized configuration space with a minimal rank greater than $O(n^2 \log n)$ or an ACC^0 circuit complexity below the lower bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "configuration space tropicalization" AND "ACC⁰ circuit complexity" - "Tensor Product problem" AND minimal rank AND tropical operations" - "constructive mapping" FROM Tensor Product TO configuration space AND ACC⁰

Top relevant hits considered: - [<http://arxiv.org/abs/1112.4251v1>] Average Case Tractability of Non-homogeneous Tensor Product Problems - [<http://arxiv.org/abs/2001.11740v1>] Exponential tractability of linear weighted tensor product problems in the worst-case setting for arbitrary linear funct - [<http://arxiv.org/abs/1708.04102v1>] Tractability of multivariate problems for standard and linear information in the worst case setting: part II - [<http://arxiv.org/abs/1607.03608v3>] On the tensor product of linear sites and Grothendieck categories - [<http://arxiv.org/abs/1808.10521v2>] Efficient quantum tensor product expanders and unitary t-designs via the zigzag product - [<http://arxiv.org/abs/2207.05809v2>] On the stability of tensor product of representations of classical groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = 1 / matrix[i][i]
            for j in range(cols):
                matrix[i][j] *= factor
            for k in range(rows):
                if k != i:
                    factor = matrix[k][i]
                    for j in range(cols):
                        matrix[k][j] -= factor * matrix[i][j]
        return matrix
```

```

def rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    row echelon_form = gaussian_elimination(matrix)
    rank = 0
    for i in range(rows):
        if any(row[i] != 0 for row in row_echelon_form):
            rank += 1
    return rank

def tensor_product(a, b):
    result = []
    for x in a:
        for y in b:
            result.append([x * y])
    return result

def tropicalize(matrix):
    rows, cols = len(matrix), len(matrix[0])
    tropicalized = [[max(matrix[i][j], matrix[j][i]) for j in range(cols)] for i in range(rows)]
    return tropicalized

n = random.randint(5, 40)
a = [random.randint(-10, 10) for _ in range(n)]
b = [random.randint(-10, 10) for _ in range(n)]

tensor_result = tensor_product(a, b)
tropicalized_tensor = tropicalize(tensor_result)

minimal_rank = rank(tropicalized_tensor)
conjecture_holds = minimal_rank <= n**2 * math.log(n)
counterexample = "" if conjecture_holds else f"Minimal rank {minimal_rank} exceeds 0({n**2 * n})"

return {
    "metric_name": "Minimal Rank",
    "metric_value": minimal_rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

true': 20, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 12, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 16, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 18, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 12, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 15, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 15, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 14, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 24, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 20, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 18, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 25, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 18, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 14, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'tropical_rank', 'metric_value': 24, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
RESULT: SUPPORTED mean=19.133333333333333 std=7.172788083366808 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The empirical test only includes 6 instances, which is too small to draw a definitive conclusion about the conjecture's validity. The metric may not scale trivially with n , and there could be larger instances where the conjecture does not hold.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: The test results indicate that for all tested instances, the tropicalized configuration space has a minimal rank within the expected bounds of $O(n^2)$ | next: Further testing with larger instance sizes is recommended to confirm the scalability and generalizability of the findings.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14682
2	preregistration	ollama_remote	glm4:latest	0	0	10099
3	novelty	ollama_remote	glm4:latest	0	0	8368
4	novelty	ollama_remote	glm4:latest	0	0	9598
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18209
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9350
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8121
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9428
9	critic	ollama_remote	glm4:latest	0	0	14087
10	judge	ollama_remote	glm4:latest	0	0	9181

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 111123 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/7d02dff3d32a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7d02dff3d32a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7d02dff3d32a.tar.gz` (if generated)